MON APPRENTISSAGE — SERIES PYTHON

Python

Du zéro à l'expert

Chapitre 28 — Python en Production

Logging, configuration, packaging, déploiement — le code professionnel

Auteur :

KETOTSA AMÉVI CLAUDE

Date de publication :

26 mars 2026

Publié dans le cadre de mon parcours d'apprentissage Python

Table des matières

Chapitre 28 — Python en Production **2**
0.1 Logging — Tracer l'exécution 2
0.2 Variables d'environnement et configuration 4
0.3 argparse — Scripts en ligne de commande 6
0.4 Packaging — Distribuer son code 8
0.5 Docker — Conteneuriser une application Python 9
0.6 subprocess — Exécuter des commandes système 11
0.7 Ce qui m'a le plus surpris dans ce chapitre 13
0.8 Résumé du Chapitre 28 13

Chapitre 28 — Python en Production

À retenir

Écrire du code qui fonctionne sur sa machine est une chose. Le déployer en production — fiable, configurable, observable, maintenable — en est une autre. Ce chapitre couvre les pratiques essentielles du développeur Python professionnel : **logging** structuré, **configuration** par variables d'environnement, **argparse** pour les scripts CLI, **packaging** avec `pyproject.toml`, et **déploiement** avec Docker. C'est la différence entre un script et une application.

Logging — Tracer l'exécution

logging — remplacer les print() en production

```python
import logging
import sys
from pathlib import Path
from datetime import datetime

# Configuration de base
logging.basicConfig(
    level=logging.DEBUG,
    format="%(asctime)s | %(levelname)-8s | "
           "%(name)s | %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
    handlers=[
        logging.StreamHandler(sys.stdout),
        logging.FileHandler(
            f"logs/app_{datetime.now():%Y%m%d}.log",
            encoding="utf-8"
        )
    ]
)

# Creer un logger nomme
logger = logging.getLogger("mon_app")

# Les 5 niveaux de log
logger.debug("Detail technique     DEBUG")
logger.info("Information normale     INFO")
logger.warning("Attention     WARNING")
logger.error("Erreur recuperee     ERROR")
logger.critical("Erreur fatale     CRITICAL")
```

```python
# Logger avec contexte
def traiter_etudiant(etudiant_id: int):
    log = logging.getLogger(f"mon_app.traitement")
    log.info(f"D but traitement  tudiant  {etudiant_id}")
    try:
        # ... traitement ...
        resultat = {"id": etudiant_id, "note": 15.5}
        log.info(f"Traitement OK     note={resultat['note']}")
        return resultat
    except Exception as e:
        log.error(f"Echec traitement {etudiant_id}: {e}",
                  exc_info=True)   # inclut le traceback !
        raise

traiter_etudiant(42)
```

Logging avancé — configuration par fichier

```python
import logging
import logging.config

# Configuration DICT     plus flexible que basicConfig
LOGGING_CONFIG = {
    "version": 1,
    "disable_existing_loggers": False,
    "formatters": {
        "standard": {
            "format": "%(asctime)s [%(levelname)s] "
                      "%(name)s: %(message)s"
        },
        "detaille": {
            "format": "%(asctime)s | %(levelname)-8s | "
                      "%(name)s:%(lineno)d | %(message)s",
            "datefmt": "%Y-%m-%d %H:%M:%S"
        },
        "json": {
            "()": "pythonjsonlogger.jsonlogger.JsonFormatter",
            "format": "%(asctime)s %(name)s %(levelname)s "
                      "%(message)s"
        }
    },
    "handlers": {
        "console": {
            "class":     "logging.StreamHandler",
            "formatter": "standard",
            "level":     "INFO",
            "stream":    "ext://sys.stdout"
        },
        "fichier": {
```

```
            "class":       "logging.handlers.
               RotatingFileHandler",
            "formatter":   "detaille",
            "level":       "DEBUG",
            "filename":    "logs/app.log",
            "maxBytes":    10_485_760,    # 10 Mo
            "backupCount": 5,
            "encoding":    "utf-8"
        },
        "erreurs": {
            "class":     "logging.FileHandler",
            "formatter": "detaille",
            "level":     "ERROR",
            "filename":  "logs/erreurs.log",
            "encoding":  "utf-8"
        }
    },
    "loggers": {
        "mon_app": {
            "handlers":  ["console", "fichier", "erreurs"],
            "level":     "DEBUG",
            "propagate": False
        }
    }
}

logging.config.dictConfig(LOGGING_CONFIG)
logger = logging.getLogger("mon_app")
logger.info("Application demarree")
```

Variables d'environnement et configuration

python-dotenv — configuration sécurisée

```
# pip install python-dotenv

# Fichier .env (JAMAIS committer dans Git !)
# ==========================================
# DATABASE_URL=postgresql://user:pass@localhost/db
# SECRET_KEY=ma_cle_ultra_secrete_32_chars
# DEBUG=False
# PORT=8000
# API_KEY=sk-xxxxxxxxxxxxxxxxxxxx
# MAX_WORKERS=4

from dotenv import load_dotenv
import os
```

```python
# Charger le .env
load_dotenv()    # cherche .env dans le repertoire courant

# Acceder aux variables
db_url      = os.getenv("DATABASE_URL")
secret_key  = os.getenv("SECRET_KEY")
debug       = os.getenv("DEBUG", "False").lower() == "true"
port        = int(os.getenv("PORT", "8000"))
max_workers = int(os.getenv("MAX_WORKERS", "4"))

print(f"Port        : {port}")
print(f"Debug       : {debug}")
print(f"Workers     : {max_workers}")

# Classe de configuration structuree
from dataclasses import dataclass, field
from typing import Optional

@dataclass
class Config:
    """Configuration centralisee de l'application."""
    # Base de donnees
    database_url:  str  = field(
        default_factory=lambda: os.getenv(
            "DATABASE_URL", "sqlite:///app.db"))
    # Securite
    secret_key:    str  = field(
        default_factory=lambda: os.getenv(
            "SECRET_KEY", "dev_key_change_in_prod"))
    # Serveur
    host:          str  = field(
        default_factory=lambda: os.getenv("HOST","0.0.0.0"))
    port:          int  = field(
        default_factory=lambda: int(os.getenv("PORT","8000")))
    debug:         bool = field(
        default_factory=lambda:
            os.getenv("DEBUG","False").lower()=="true")
    # Performance
    max_workers:   int  = field(
        default_factory=lambda: int(
            os.getenv("MAX_WORKERS","4")))

    def valider(self):
        if self.debug and "prod" in self.database_url:
            raise ValueError(
                "DEBUG=True interdit en production !")
        if len(self.secret_key) < 32:
            raise ValueError("SECRET_KEY trop courte !")
        return self
```

```python
config = Config().valider()
print(f"Config charg e : port={config.port}, "
      f"debug={config.debug}")
```

argparse — Scripts en ligne de commande

argparse — CLI professionnel

```python
import argparse
import sys
import logging
from pathlib import Path

def creer_parser():
    parser = argparse.ArgumentParser(
        prog="analyseur",
        description="Outil d'analyse des donnees etudiants",
        epilog="Exemple : python analyseur.py data.csv "
               "--output rapport.pdf --verbose"
    )

    # Arguments positionnels (obligatoires)
    parser.add_argument(
        "fichier",
        type=Path,
        help="Fichier CSV des donn es a analyser"
    )

    # Options (facultatives)
    parser.add_argument(
        "--output", "-o",
        type=Path,
        default=Path("rapport.html"),
        help="Fichier de sortie (defaut: rapport.html)"
    )
    parser.add_argument(
        "--format", "-f",
        choices=["html", "pdf", "csv", "json"],
        default="html",
        help="Format de sortie (defaut: html)"
    )
    parser.add_argument(
        "--note-min",
        type=float,
        default=0.0,
        help="Note minimale pour filtrer (defaut: 0)"
    )
    parser.add_argument(
```

```
41         "--verbose", "-v",
42         action="store_true",
43         help="Mode verbeux"
44     )
45     parser.add_argument(
46         "--version",
47         action="version",
48         version="%(prog)s 1.0.0"
49     )
50
51     # Groupe mutuellement exclusif
52     groupe = parser.add_mutually_exclusive_group()
53     groupe.add_argument("--admis",   action="store_true",
54                         help="Afficher seulement les admis")
55     groupe.add_argument("--recales", action="store_true",
56                         help="Afficher seulement les recales")
57
58     return parser
59
60
61 def main():
62     parser = creer_parser()
63     args   = parser.parse_args()
64
65     # Configurer le logging selon verbosite
66     niveau = logging.DEBUG if args.verbose else logging.INFO
67     logging.basicConfig(level=niveau,
68                         format="%(levelname)s: %(message)s")
69     log = logging.getLogger("analyseur")
70
71     # Valider les arguments
72     if not args.fichier.exists():
73         log.error(f"Fichier introuvable : {args.fichier}")
74         sys.exit(1)
75
76     log.info(f"Analyse de : {args.fichier}")
77     log.info(f"Sortie     : {args.output} ({args.format})")
78     log.debug(f"Note min   : {args.note_min}")
79
80     if args.admis:
81         log.info("Filtre : admis uniquement (note >= 10)")
82     elif args.recales:
83         log.info("Filtre : recales uniquement (note < 10)")
84
85     # ... traitement principal ...
86     log.info("Analyse terminee.")
87
88
89 if __name__ == "__main__":
90     main()
```

```
# Utilisation :
# python analyseur.py data.csv
# python analyseur.py data.csv -o rapport.pdf -f pdf -v
# python analyseur.py data.csv --admis --note-min 12
```

Packaging — Distribuer son code

pyproject.toml — structure moderne

```
% Structure d'un package Python professionnel :
%
% mon_package/
%   src/
%     mon_package/
%       __init__.py
%       core.py
%       utils.py
%       cli.py
%   tests/
%     __init__.py
%     test_core.py
%     test_utils.py
%   docs/
%   pyproject.toml            configuration moderne
%   README.md
%   LICENSE
%   .gitignore
%   .env.example              mod le sans secrets
%   Dockerfile

% Contenu de pyproject.toml :
% ============================
% [build-system]
% requires = ["setuptools>=68", "wheel"]
% build-backend = "setuptools.backends.legacy:build"
%
% [project]
% name = "mon-package"
% version = "1.0.0"
% description = "Description courte du package"
% readme = "README.md"
% license = {file = "LICENSE"}
% authors = [{name = "Alice Dupont",
%             email = "alice@exemple.com"}]
% keywords = ["data", "analyse", "statistiques"]
% classifiers = [
%     "Programming Language :: Python :: 3",
```

```
%     "License :: OSI Approved :: MIT License",
%     "Operating System :: OS Independent",
% ]
% requires-python = ">=3.10"
% dependencies = [
%     "pandas>=2.0",
%     "numpy>=1.24",
%     "requests>=2.28",
%     "python-dotenv>=1.0",
% ]
%
% [project.optional-dependencies]
% dev  = ["pytest>=7.0", "black", "mypy", "ruff"]
% docs = ["sphinx", "sphinx-rtd-theme"]
%
% [project.scripts]
% analyseur = "mon_package.cli:main"
%
% [tool.pytest.ini_options]
% testpaths = ["tests"]
% addopts   = "-v --tb=short"
%
% [tool.black]
% line-length = 88
% target-version = ["py310"]
%
% [tool.ruff]
% line-length = 88
% select = ["E","W","F","I"]

% Commandes essentielles :
% pip install -e .                installation dev (editable)
% pip install -e ".[dev]"         avec d pendances dev
% python -m build                 construire la distribution
% pip install dist/*.whl          installer localement
% twine upload dist/*             publier sur PyPI
```

Docker — Conteneuriser une application Python

Dockerfile et docker-compose

```
% ==============================================================
% Dockerfile     image Docker pour une app FastAPI
% ==============================================================
%
% FROM python:3.11-slim
%
% # Metadonnees
```

```
% LABEL maintainer="alice@exemple.com"
% LABEL version="1.0.0"
%
% # Variables d'environnement
% ENV PYTHONDONTWRITEBYTECODE=1 \
%     PYTHONUNBUFFERED=1        \
%     PORT=8000
%
% # Repertoire de travail
% WORKDIR /app
%
% # Installer les dependances en premier (cache Docker)
% COPY requirements.txt .
% RUN pip install --no-cache-dir -r requirements.txt
%
% # Copier le code source
% COPY src/ ./src/
% COPY .env.example .env
%
% # Utilisateur non-root pour la s curite
% RUN adduser --disabled-password --gecos "" appuser
% USER appuser
%
% # Exposer le port
% EXPOSE $PORT
%
% # Commande de demarrage
% CMD ["uvicorn", "src.main:app",
%      "--host", "0.0.0.0",
%      "--port", "8000",
%      "--workers", "2"]

% ============================================================
% docker-compose.yml     orchestrer plusieurs services
% ============================================================
%
% version: "3.9"
%
% services:
%   api:
%     build: .
%     ports:
%       - "8000:8000"
%     environment:
%       - DATABASE_URL=postgresql://user:pass@db:5432/app
%       - SECRET_KEY=${SECRET_KEY}
%       - DEBUG=False
%     depends_on:
%       db:
%         condition: service_healthy
```

```
%      restart: unless-stopped
%      volumes:
%        - ./logs:/app/logs
%
%    db:
%      image: postgres:15-alpine
%      environment:
%        POSTGRES_USER:     user
%        POSTGRES_PASSWORD: pass
%        POSTGRES_DB:       app
%      volumes:
%        - postgres_data:/var/lib/postgresql/data
%      healthcheck:
%        test: ["CMD", "pg_isready", "-U", "user"]
%        interval: 10s
%        timeout:  5s
%        retries:  5
%
% volumes:
%   postgres_data:

% Commandes Docker essentielles :
% docker build -t mon-app .
% docker run -p 8000:8000 --env-file .env mon-app
% docker-compose up -d
% docker-compose logs -f api
% docker-compose down
```

subprocess — Exécuter des commandes système

subprocess — piloter le système

```python
import subprocess
import sys

# Executer une commande simple
resultat = subprocess.run(
    ["python", "--version"],
    capture_output=True,
    text=True
)
print(f"Version : {resultat.stdout.strip()}")
print(f"Code retour : {resultat.returncode}")

# Avec verification automatique des erreurs
try:
    subprocess.run(
        ["git", "status"],
```

```
        check=True,         # l ve si returncode != 0
        capture_output=True,
        text=True
    )
except subprocess.CalledProcessError as e:
    print(f"Erreur git : {e.stderr}")
except FileNotFoundError:
    print("git non installe ou introuvable")

# Lire la sortie
proc = subprocess.run(
    ["pip", "list", "--format=json"],
    capture_output=True, text=True, check=True
)
import json
packages = json.loads(proc.stdout)
print(f"Packages installes : {len(packages)}")
for pkg in packages[:5]:
    print(f"  {pkg['name']:30} {pkg['version']}")

# Pipeline     enchainer des commandes
def compter_lignes_python(dossier="."):
    """Compte les lignes de code Python dans un dossier."""
    try:
        find = subprocess.Popen(
            ["find", dossier, "-name", "*.py",
             "-not", "-path", "*/.*"],
            stdout=subprocess.PIPE
        )
        resultat = subprocess.run(
            ["xargs", "wc", "-l"],
            stdin=find.stdout,
            capture_output=True, text=True
        )
        find.stdout.close()
        lignes = resultat.stdout.strip().split("\n")
        if lignes:
            total = lignes[-1].strip().split()[0]
            print(f"Total lignes Python : {total}")
    except Exception as e:
        print(f"Erreur : {e}")

compter_lignes_python(".")
```

Ce qui m'a le plus surpris dans ce chapitre

Ce qui m'a résisté

Trois révélations de ce chapitre :

1. `print()` en production est une erreur de débutant. Quand un programme tourne sur un serveur sans terminal interactif, les `print()` disparaissent dans le vide. Le logging écrit dans des fichiers rotatifs, filtrables par niveau, horodatés, exploitables par des outils comme ELK Stack ou Datadog. La transition de `print()` vers `logger.info()` n'est pas cosmétique — c'est la différence entre un code qu'on peut observer et un code qu'on surveille à l'aveugle.

2. Les secrets ne vont JAMAIS dans le code source. Un `API_KEY = "sk-xxxx"` hardcodé dans le code, même dans un dépôt privé, est une faille de sécurité. Les variables d'environnement via `.env` + `python-dotenv` permettent de séparer la configuration du code. Le fichier `.env` ne va jamais dans Git — seul `.env.example` (sans valeurs réelles) est commité. Cette règle est absolue et non négociable en environnement professionnel.

3. Docker résout le problème "ça marche sur ma machine". Conteneuriser une application Python avec Docker garantit que l'environnement d'exécution est identique partout — développement, test, production. Plus de "j'ai Python 3.9 mais le serveur a Python 3.11". Plus de dépendances manquantes. Le Dockerfile est la documentation exécutable de l'environnement. Pour un Data Scientist qui déploie des modèles ML, c'est une compétence incontournable.

Résumé du Chapitre 28

Ce que j'ai appris dans ce chapitre :

- **logging** : niveaux DEBUG/INFO/WARNING/ERROR, handlers, rotation.
- `exc_info=True` — inclure le traceback dans les logs d'erreur.
- **Variables d'environnement** : `.env` + `python-dotenv`.
- Classe `Config` — centraliser la configuration de l'application.
- **argparse** : CLI professionnel avec arguments, options, groupes.
- **pyproject.toml** — packaging moderne, dépendances, scripts CLI.
- **Dockerfile** — conteneuriser une application Python.
- **docker-compose** — orchestrer API + base de données.
- **subprocess** — piloter des commandes système depuis Python.

"Un code en production n'est pas un code qui tourne. C'est un code qu'on peut observer, configurer, déployer et maintenir — sans stress."